SIMATS ENGINEERING

CSA09 – PROGRAMMING IN JAVA

ASSIGNMENT 5 – CO5

Data Persistence Engineering – Files and Databases (Java I/O, Serialization, JDBC)

Name	Sanjay Adithya S J
Register Number	192311336
Course Code	CSA0921
Course Outcome	CO5 – Java I/O, serialization, JDBC
GitHub Repository	https://github.com/sanoryx/CSA0921-JAVA

AIM AND THEORY

Aim: To decide where and how data is stored (text file, binary stream, serialization or relational database), and to make the program correct, safe and efficient as the data grows – following Model → Choose a Store → Design → Protect → Implement → Measure → Compare → Decide. **Theory:** Java I/O has *byte streams* (InputStream/OutputStream, FileInputStream), *character streams* (Reader/Writer, FileReader) and *buffered streams* (BufferedInputStream, BufferedReader/Writer) that read or write a block at a time, reducing the number of operating-system calls. *Serialization* (Serializable, ObjectOutputStream, transient, serialVersionUID) saves an object graph. **JDBC** connects a program to a database through a driver: Statement (plain SQL), PreparedStatement (precompiled, parameterised – safe from SQL injection), CallableStatement (stored procedures), transactions (setAutoCommit(false), commit, rollback), batch processing (addBatch/executeBatch) and metadata (ResultSetMetaData, DatabaseMetaData). **Environment note:** all programs were executed on OpenJDK 21. A MySQL server was not available in the sandbox, so every JDBC program was run with the same JDBC code against **SQLite** (driver sqlite-jdbc 3.36.0.3); the SQL scripts for MySQL are given and only the connection URL changes. SQLite is a local file database, so absolute times (especially for autocommit, where every row is flushed to disk) differ from a networked MySQL server; the *trends* are the evidence. The pass file and the 50 MB logs are generated by the programs.

PART A – PERSISTENCE DESIGN AND PSEUDOCODE (25 marks)

TASK A1 – Design a File Format: Transport Office Bus-Pass Records (7 marks)

A1.1 Represent the record (2)

	(a) Delimited text file	(b) Binary file with DataOutputStream
Field order	passId, regNo, name, routeNo, boardingStop, validityDate, amountPaid	same order
Layout	one record per line, fields separated by **,** (comma); example: `P1001,192311001,Arun,12,Anna Nagar,2026-12-31,4500.0`	writeUTF(passId); writeUTF(regNo); writeUTF(name); writeInt(route); writeUTF(stop); writeLong(validityEpochDay); writeDouble(amount) – UTF strings are length-prefixed (2-byte length), numbers have fixed size
Field sizes	variable (about 60 bytes per record, 8,000 records ≈ 0.5 MB)	variable strings; for random access use fixed sizes: passId 8 bytes, regNo 9, name 30, route 4, stop 30, date 8, amount 8 = 97 bytes padded to 100
Name containing the delimiter (e.g. “Raj, Kumar”)	the field is wrapped in double quotes and a quote inside is doubled (CSV rule), or the comma is escaped as **\,**; the reader must understand the rule; simplest alternative: choose a delimiter that cannot occur in names, e.g. **|**, and reject names that contain it	no problem – length-prefixed strings never use a delimiter

A1.2 Design the operations (3)

Stream choices: FileWriter/BufferedWriter (character stream + buffer) in *append mode* for add – text data, one write per line and the buffer flushes at close; **FileReader/BufferedReader** for find and list – readLine() returns a whole record and the buffer avoids one OS call per character. FileInputStream/FileOutputStream (byte streams) are used only for the binary design or non-text data; they would force manual decoding of characters.

Pseudocode – add a pass

```
ALGORITHM AddPass(file, pass)
BEGIN
  IF pass.name CONTAINS delimiter THEN name <- quote(pass.name)
  line <- join(pass fields, ",")
  OPEN writer <- BufferedWriter(FileWriter(file, append = TRUE))      // creates the file if it is missing
  TRY    writer.write(line); writer.newLine()
  FINALLY CLOSE writer                                                // flushes the buffer
END
```

Pseudocode – find a pass by ID

```
ALGORITHM FindPass(file, passId)
BEGIN
  OPEN reader <- BufferedReader(FileReader(file))
  TRY
    WHILE (line <- reader.readLine()) is not null DO
      fields <- split(line, ",")
      IF count(fields) <> 7 THEN CONTINUE               // damaged line: skip (and count it)
      IF fields[0] = passId THEN RETURN fields
    END WHILE
    RETURN NOT_FOUND
  FINALLY CLOSE reader
END
```

Pseudocode – list passes of a route

```
ALGORITHM ListByRoute(file, route)
BEGIN
  result <- empty list
  OPEN reader; TRY
    WHILE (line <- reader.readLine()) is not null DO
      fields <- split(line, ",")
      IF count(fields) = 7 AND isInteger(fields[3]) AND integer(fields[3]) = route THEN result.add(fields)
    END WHILE
  FINALLY CLOSE reader
  RETURN result
END
```

A1.3 Design for damaged data (2)

- **Damaged line:** every line is validated on reading – correct field count, numeric route/amount, valid date; an invalid line is skipped, counted and logged with its line number so that it never stops the search (implemented and tested in B1/B2).
- **Missing file:** FileNotFoundException/NoSuchFileException is caught: for *add* the file is created (CREATE option); for *find/list* an empty result is returned and the office is told “no pass file yet”.
- **Incomplete write after power failure:** the last line may be cut off → it fails the field-count check and is skipped; to avoid losing a good record write the line, flush and sync (FileChannel.force); for safety write a new file *passes.tmp* and atomically rename it (Files.move ATOMIC_MOVE), or keep an end-of-record marker/checksum per line.

Alternative concept: RandomAccessFile with fixed-length records

Record i lives at byte offset **i × recordSize**; `raf.seek(i*100); raf.readFully(buf)`. **Find by pass ID:** if the pass ID is a number (or has a known numeric part) the record index is computed and the lookup is **O(1)** (one seek + one read); otherwise a scan is still O(n), exactly like the text file, unless an index (HashMap id → position) is kept. The delimited file always needs an **O(n)** scan (measured 17.4 ms for 100,000 records, B1.4). **Wasted space:** every field is padded to its maximum size: e.g. an average record of 60 bytes stored in 100 bytes wastes 40 bytes (40 %); for 8,000 passes = 320 KB wasted of 800 KB – O(n × (max − average)) – acceptable for a small file. Updating a record in place is easy; deleting leaves holes; a damaged record does not shift the following ones (positions stay valid), which also improves damage containment.

TASK A2 – Don’t Read What You Don’t Need (5 marks)

A2.1 Predict the cost (2)

FileInputStream.read() returns one byte per call, so a 50 MB (52,428,800-byte) file needs **one read call per byte = about 52.4 million calls**, and each call is a system call into the operating system (context switch, kernel buffer copy). The overhead per call (hundreds of nanoseconds) dominates the useful work (one comparison with ‘\n’): I predict about **10 s** for 50 MB with time growing linearly with the file size: 1 MB ≈ 0.2 s, 10 MB ≈ 2 s, 50 MB ≈ 10 s.

A2.2 Redesign using buffering (2)

Pseudocode – count lines with a byte-array buffer

```
ALGORITHM CountLines(file)
BEGIN
  buffer <- new byte[65536]; count <- 0
  OPEN in <- FileInputStream(file)
  TRY
    WHILE (k <- in.read(buffer)) <> -1 DO          // one OS call returns up to 65,536 bytes
```

```
      FOR i <- 0 TO k-1 DO IF buffer[i] = '\n' THEN count <- count + 1
    END WHILE
  FINALLY CLOSE in
  RETURN count
END
```

A BufferedInputStream (8 KB default) or BufferedReader.readLine() does the same internally: the program still makes one read() call per byte *from the buffer*, but the buffer fetches 8,192 bytes from the OS at a time. The number of operating-system calls drops from 52.4 million to **52,428,800 / 8,192 = 6,400** (or 800 with a 64 KB buffer) – a reduction by a factor equal to the buffer size.

A2.3 When buffering does not help (1)

When the program already reads in large blocks (read(byte[65536])) – an extra BufferedInputStream just copies the data twice; or when it reads a few bytes at random positions (RandomAccessFile, a single header read) or the data is already in memory; also for slow sources where the device (network, disk) is the bottleneck, not the number of calls.

Alternative concept: Files.readAllLines()

`Files.readAllLines(path).size()` reads the whole file at once and builds a List<String>. **Time:** O(n) in the file size, like all approaches – measured 325 ms for 50 MB (B1.3). **Memory:** O(file size) – for 50 MB of text roughly 100–150 MB of heap for 1.5 million String objects – versus **O(buffer size)** (8 KB–64 KB) for the buffered stream/BufferedReader.readLine() loop. readAllLines is appropriate for small/medium files (a few MB) that are processed several times; a buffered streaming approach is appropriate for large files (hundreds of MB, logs) where one pass is enough and memory is limited.

TASK A3 – Discover the Data-Access Design Pattern (5 marks)

A3.1 Identify the common pattern (2)

Repeated in StudentDB, CourseDB and MarksDB	Different in each
1 open a connection; 2 create the statement; 3 set parameters; 4 execute; 5 loop over the ResultSet; 6 close ResultSet/Statement/Connection; 7 catch and report SQLException	the SQL text; the parameters; the way a ResultSet row is converted into a Student, Course or Marks object; the table and column names

A3.2 Generic data-access interface (2)

Pseudocode – Dao and StudentDao

```
INTERFACE Dao<T, K>
   insert(T entity)            : void
   findById(K id)              : T or NONE
   findAll()                   : List<T>
   update(T entity)            : void
   delete(K id)                : void
END INTERFACE

CLASS StudentDao IMPLEMENTS Dao<Student, String>
   insert(s)    : prepare "INSERT INTO student(reg_no,name,dept,cgpa) VALUES (?,?,?,?)"; set 4 params; execute
   findById(id) : prepare "SELECT ... FROM student WHERE reg_no = ?"; RETURN mapRow(result) or NONE
   findAll()    : prepare "SELECT ... FROM student"; FOR EACH row ADD mapRow(row) to list; RETURN list
   update(s)    : prepare "UPDATE student SET name=?, dept=?, cgpa=? WHERE reg_no=?"
   delete(id)   : prepare "DELETE FROM student WHERE reg_no = ?"
   mapRow(row)  : RETURN Student(row.reg_no, row.name, row.dept, row.cgpa)       // the only row-specific code
END CLASS
(the repeated open/prepare/execute/close steps live once in a helper: query(sql, params, rowMapper))
```

A3.3 Map objects to tables (1)

Student field	Java type	Column	SQL type / constraint
regNo	String	reg_no	VARCHAR(9) **PRIMARY KEY**
name	String	name	VARCHAR(60) NOT NULL
dept	String	dept	VARCHAR(30)
cgpa	double	cgpa	DECIMAL(4,2) CHECK (cgpa BETWEEN 0 AND 10)

Alternative concept: SQL written directly in the service or GUI classes

The SQL strings are scattered across n service/GUI classes. If a column is renamed (cgpa → gpa) **every class that contains that SQL must be found and edited** – c places (for example 12), and a missed one fails only at run time; with Dao<T,K> the column name exists in **one place** (StudentDao). **Time and space per operation:** identical – both send the same SQL and read the same rows (O(rows) time, O(rows) objects); the Dao adds one method call and possibly a RowMapper object (a few nanoseconds, a few bytes), which is negligible next to a database round trip of milliseconds. The Dao design costs more classes up front but saves maintenance and enables testing with a fake Dao.

TASK A4 – What Should Be Serialized? (4 marks)

A4.1 Decide what to persist (2)

- **transient:** the live database **Connection** (not Serializable – an attempt throws NotSerializableException, shown below; it is also meaningless after restart) and the **password** (a secret must not be written in clear text to a file). name and register number are persisted.
- **Object graph:** serializing the Hostel object writes the Hostel, then recursively every reachable object: the list of Rooms, each Room's list of Residents; all classes in the graph must implement Serializable; shared references are written once and restored as shared references; transient and static fields are skipped; on reading, transient fields get default values (null/0) and constructors of Serializable classes are **not** run.

Execution and output – non-serializable field in the graph:

```
$ java NS   (a Resident that holds a Connection-like non-serializable field)
FAILED: java.io.NotSerializableException: NS$Conn
```

A4.2 Plan for change (2)

A phone number field is added after files were saved. **With an explicit serialVersionUID (e.g. 1L) kept the same:** the old file can still be read; the new field is not in the stream, so it receives the **default value (null)** – not the initialiser value ("unknown"), because field initialisers do not run during deserialization. **Without serialVersionUID:** the JVM computes it from the class structure; adding a field changes it, the stored value no longer matches and readObject throws **InvalidClassException** – the old files become unreadable.

Execution and output (real run of the two class versions):

```
$ java -cp v1 Save; java -cp v2 Load / java -cp v0 Save; java -cp v3 Load
--- explicit serialVersionUID=1L, old file written by v1, read by v2 (phone added) ---
loaded: name=Arun regNo=192311001 password=null phone=null
--- NO serialVersionUID, old file written by v0, read after adding phone (v3) ---
FAILED: java.io.InvalidClassException: Resident; local class incompatible: stream classdesc serialVersionUID =
    7860459363859177249, local class serialVersionUID = -4150196601207458118
--- transient field after loading ---
loaded: name=Arun regNo=192311001 password=null
```

Alternative concept: CSV or JSON instead of Java serialization

File size (measured for n residents with 2 fields): serialization 29,020 / 299,020 / 3,099,020 bytes for n = 1,000 / 10,000 / 100,000 versus CSV 18,890 / 198,890 / 2,098,890 bytes – serialization is about 1.5× larger (class metadata and type markers); JSON would be larger than CSV (field names repeated in every object). **Time to save/load** (ms, best of 4): serialization 1.2/1.9, 12.5/15.5, 94.2/90.4; CSV 1.9/0.9, 6.4/9.3, 20.0/58.9 – both O(n); CSV is 1.5–5× faster here. **New field:** serialization depends on serialVersionUID and gives null/0 defaults (above); CSV/JSON are tolerant – JSON ignores unknown/missing fields, CSV needs a header or column convention; both are readable by other languages and tools, serialization is Java-only and has security risks when reading untrusted data.

TASK A5 – Design for Correctness and Safety (4 marks)

A5.1 Transaction design (2)

Pseudocode – transfer Rs 500 from hostel wallet to canteen wallet

```
ALGORITHM Transfer(con, fromWallet, toWallet, amount)
BEGIN
  oldAuto <- con.autoCommit; con.autoCommit <- FALSE            // start the transaction
  TRY
    n1 <- EXECUTE "UPDATE wallet SET balance = balance - ? WHERE id = ? AND balance >= ?" (amount, fromWallet, amount)
    IF n1 <> 1 THEN THROW InsufficientFunds                      // nothing was changed
    n2 <- EXECUTE "UPDATE wallet SET balance = balance + ? WHERE id = ?" (amount, toWallet)
    IF n2 <> 1 THEN THROW UnknownWallet
    con.commit()                                                 // both updates become permanent together
  CATCH ANY exception e
    con.rollback()                                               // undo the first update: no money lost or created
    THROW e
  FINALLY con.autoCommit <- oldAuto
END
```

If the program fails between the two updates, no commit has happened, so the database (or the rollback in the catch block) discards the debit; money is neither lost nor created (atomicity).

A5.2 Injection analysis (2)

For `String sql = "SELECT * FROM users WHERE id='" + id + "' AND pwd='" + pwd + "'"` the input is pasted into the SQL text. With **pwd =** `x' OR '1'='1` the statement becomes `... WHERE id='arun' AND pwd='x' OR '1'='1';` AND binds tighter than OR, so the condition is true for every row and the query succeeds without the password. With **id =** `admin'--` the rest of the line becomes a comment and the password test disappears.

Pseudocode – safe login with PreparedStatement

```
ALGORITHM Login(con, id, pwd)
BEGIN
  ps <- con.prepareStatement("SELECT 1 FROM users WHERE id = ? AND pwd_hash = ?")
  ps.setString(1, id); ps.setString(2, hash(pwd))               // values are DATA, never SQL
  RETURN ps.executeQuery().next()
END
```

The parameters are sent separately from the SQL text, so quote characters in the input are just characters of a value. (A real system also stores a salted hash instead of the password.)

Alternative concept: two separate auto-commit statements

Each UPDATE is committed immediately. If the program fails between them, the hostel wallet is already debited by Rs 500 and committed, but the canteen wallet is never credited: **Rs 500 is lost** and the database stays inconsistent. **Time cost:** a transaction costs one commit (one flush) for both updates; two auto-commits cost two commits – measured in B3.3: 10,000 batched rows in one transaction took 34 ms, with auto-commit on 6,656 ms (about 200× more) – so the transaction is both safer *and* usually faster. The only cost of a transaction is that locks are held until commit (shorter than a human reaction time for two updates).

PART B – IMPLEMENTATION AND EXPERIMENTAL INVESTIGATION (25 marks)

TASK B1 – Implement and Instrument the File Store (7 marks)

Complete the Code – PassStore (TODO 1)

Source file: PassStore.java

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;
import java.util.Arrays;

public class PassStore {
    private final Path file;
    PassStore(Path file) { this.file = file; }
    public void add(String passId, String regNo, String name, int route, double amount)
            throws IOException {
        try (BufferedWriter w = Files.newBufferedWriter(file,
                StandardOpenOption.CREATE, StandardOpenOption.APPEND)) {
            w.write(String.join(",", passId, regNo, name,
                    String.valueOf(route), String.valueOf(amount)));
            w.newLine();
        }
    }

    /** Returns the five fields of the pass with the given ID, or null if not found. */
    public String[] find(String passId) throws IOException {
        try (BufferedReader r = Files.newBufferedReader(file)) {          // TODO 1 completed
            String line;
            while ((line = r.readLine()) != null) {
                String[] f = line.split(",", -1);
                if (f.length != 5) continue;                              // damaged line: skip it
                if (f[0].equals(passId)) return f;
            }
        }
        return null;
    }

    /** list all passes on a route (operation iii); damaged lines are skipped */
    public java.util.List<String[]> listByRoute(int route) throws IOException {
        java.util.List<String[]> out = new java.util.ArrayList<>();
        try (BufferedReader r = Files.newBufferedReader(file)) {
            for (String line; (line = r.readLine()) != null; ) {
                String[] f = line.split(",", -1);
                if (f.length != 5) continue;
                try { if (Integer.parseInt(f[3]) == route) out.add(f); } catch (NumberFormatException e) { /* damaged */ }
            }
        }
        return out;
    }
    public static void main(String[] args) throws IOException {
        Path f = Files.createTempFile("passes", ".csv");
        PassStore store = new PassStore(f);
        store.add("P1001", "192311001", "Arun", 12, 4500);
        Files.writeString(f, "P1002,DAMAGED LINE\n", StandardOpenOption.APPEND);
        store.add("P1003", "192311003", "Divya", 7, 3800);
        System.out.println(Arrays.toString(store.find("P1003")));
        System.out.println(Arrays.toString(store.find("P1002")));
        Files.delete(f);
    }
}
```

Execution and output (matches the expected output):

```
$ javac PassStore.java && java PassStore
[P1003, 192311003, Divya, 7, 3800.0]
null
```

Why the damaged line for P1002 does not stop the search: find() reads line by line; the damaged line "P1002,DAMAGED LINE" splits into 2 fields, so the check *f.length != 5* makes the loop **continue** with the next line instead of failing; P1003 is found after it. Because the damaged line is skipped, the search for P1002 returns null (it is treated as not present).

Source file: B1Tests.java

```
import java.io.*;
import java.nio.file.*;
import java.util.*;

public class B1Tests {
    static long sink;
    static int countUnbuffered(Path p) throws IOException { int n = 0; try (InputStream in = new FileInputStream(p.toFile()))
         { int b; while ((b = in.read()) != -1) if (b == '\n') n++; } return n; }
    static int countBuffered(Path p) throws IOException { int n = 0; try (InputStream in = new BufferedInputStream(new
        FileInputStream(p.toFile()), 8192)) { int b; while ((b = in.read()) != -1) if (b == '\n') n++; } return n; }
    static int countArray(Path p) throws IOException { int n = 0; byte[] buf = new byte[65536]; try (InputStream in = new
        FileInputStream(p.toFile())) { int k; while ((k = in.read(buf)) != -1) for (int i = 0; i < k; i++) if (buf[i] ==
        '\n') n++; } return n; }
    static int countReader(Path p) throws IOException { int n = 0; try (BufferedReader r = Files.newBufferedReader(p)) {
        while (r.readLine() != null) n++; } return n; }
    static int countAll(Path p) throws IOException { return Files.readAllLines(p).size(); }

    public static void main(String[] args) throws Exception {
        PassStore st; Path f = Files.createTempFile("bp", ".csv"); st = new PassStore(f);
        System.out.println("== B1.1 test table ==");
        st.add("P1", "192311001", "Arun", 12, 4500); st.add("P2", "192311002", "Meena", 12, 4000); st.add("P3", "192311003",
            "Ravi", 7, 3800);
        System.out.println("T1 normal add+find P2        -> " + Arrays.toString(st.find("P2")));
        System.out.println("T2 pass ID not present P999  -> " + Arrays.toString(st.find("P999")));
        Files.writeString(f, "P4,only,three\n\nP5,1,N,9,abc,extra\nP6,192311006,Lata,12,xx\n", StandardOpenOption.APPEND);
        st.add("P7", "192311007", "Kumar", 12, 4100);
        System.out.println("T3 damaged lines present, find P7 -> " + Arrays.toString(st.find("P7")));
        System.out.println("T4 find damaged P4           -> " + Arrays.toString(st.find("P4")));
        System.out.print("T5 list route 12             -> "); for (String[] r : st.listByRoute(12)) System.out.print(r[0] + "
             "); System.out.println(" (P6 has 5 fields, route ok, bad amount is NOT checked by list; P1,P2,P6,P7)");
        Files.delete(f);

        System.out.println("\n== B1.3 line counting (ms; best of 3 for buffered variants, 1 run for the slow unbuffered 50
            MB) ==");
        System.out.printf("%-8s %-12s %-14s %-12s %-14s %-14s %-10s%n", "size", "reads(unbuf)", "unbuffered", "buffered",
            "byte[64K]", "BufferedReader", "readAllLines");
        for (int mb : new int[]{1, 10, 50}) {
            Path p = Files.createTempFile("log", ".txt");
            try (BufferedWriter w = Files.newBufferedWriter(p)) { Random r = new Random(1); long bytes = 0; while (bytes < mb
                 * 1024L * 1024L) { String l = "2026-11-06,19231" + (1000 + r.nextInt(9000)) + ",PRESENT,GATE" +
                r.nextInt(5); w.write(l); w.newLine(); bytes += l.length() + 1; } }
            long size = Files.size(p);
            double u = 1e18, b = 1e18, a = 1e18, rd = 1e18, al = 1e18; int lines = 0;
            for (int rep = 0; rep < (mb == 50 ? 1 : 3); rep++) { long s = System.nanoTime(); lines = countUnbuffered(p); u =
                Math.min(u, (System.nanoTime() - s) / 1e6); }
            for (int rep = 0; rep < 3; rep++) { long s = System.nanoTime(); sink += countBuffered(p); b = Math.min(b,
                (System.nanoTime() - s) / 1e6); s = System.nanoTime(); sink += countArray(p); a = Math.min(a,
                (System.nanoTime() - s) / 1e6); s = System.nanoTime(); sink += countReader(p); rd = Math.min(rd,
                (System.nanoTime() - s) / 1e6); s = System.nanoTime(); sink += countAll(p); al = Math.min(al,
                (System.nanoTime() - s) / 1e6); }
            System.out.printf("%-8s %-12d %-14.0f %-12.0f %-14.1f %-14.0f %-10.0f  lines=%d%n", mb + " MB", size, u, b, a,
                rd, al, lines);
            Files.delete(p);
        }
        System.out.println("\n== B1.4 find() time as the file grows (ms, average of 50 searches for the LAST record = worst
            case) ==");
        System.out.printf("%-10s %-14s %-14s %-16s%n", "records", "file scan", "HashMap lookup", "HashMap build");
        for (int n : new int[]{1000, 5000, 10000, 50000, 100000}) {
            Path p = Files.createTempFile("pf", ".csv"); PassStore s = new PassStore(p);
            try (BufferedWriter w = Files.newBufferedWriter(p)) { for (int i = 0; i < n; i++) { w.write("P" + i + ",19231" +
                i + ",Name" + i + "," + (i % 20) + "," + (3000 + i % 500) + ".0"); w.newLine(); } }
            for (int i = 0; i < 5; i++) sink += s.find("P" + (n - 1)).length;
            long t = System.nanoTime(); for (int i = 0; i < 50; i++) sink += s.find("P" + (n - 1 - i)).length; double scan =
                (System.nanoTime() - t) / 1e6 / 50;
            t = System.nanoTime(); Map<String, String[]> m = new HashMap<>(); try (BufferedReader r =
                Files.newBufferedReader(p)) { for (String l; (l = r.readLine()) != null; ) { String[] x = l.split(",", -1);
                if (x.length == 5) m.put(x[0], x); } } double build = (System.nanoTime() - t) / 1e6;
            t = System.nanoTime(); for (int i = 0; i < 100000; i++) sink += m.get("P" + (i % n)).length; double look =
                (System.nanoTime() - t) / 1e6 / 100000;
            System.out.printf("%-10d %-14.3f %-14.5f %-16.1f%n", n, scan, look, build);
            Files.delete(p);
        }
        System.out.println("(sink " + (sink != 0) + ")");
    }
}
```

Execution and output:

```
$ javac B1Tests.java && java B1Tests
== B1.1 test table ==
T1 normal add+find P2        -> [P2, 192311002, Meena, 12, 4000.0]
T2 pass ID not present P999  -> null
T3 damaged lines present, find P7 -> [P7, 192311007, Kumar, 12, 4100.0]
T4 find damaged P4           -> null
T5 list route 12             -> P1 P2 P6 P7  (P6 has 5 fields, route ok, bad amount is NOT checked by list; P1,P2,P6,P7)

== B1.3 line counting (ms; best of 3 for buffered variants, 1 run for the slow unbuffered 50 MB) ==
size     reads(unbuf) unbuffered     buffered     byte[64K]      BufferedReader readAllLines
1 MB     1048600      237            21           1.5            9              7           lines=29960
10 MB    10485790     2520           185          4.4            40             28          lines=299594
50 MB    52428810     12020          779          17.5           76             325         lines=1497966

== B1.4 find() time as the file grows (ms, average of 50 searches for the LAST record = worst case) ==
records    file scan      HashMap lookup HashMap build
1000       0.680          0.00075        3.8
5000       3.092          0.00002        1.4
10000      2.198          0.00021        11.1
50000      10.534         0.00022        34.0
100000     17.446         0.00006        28.0
(sink true)
```

B1.1 Validate the implementation (2)

Test	Input	Result	What it verifies
T1 normal	add P1,P2,P3; find P2	[P2, 192311002, Meena, 12, 4000.0]	add appends correctly; find returns the 5 fields
T2 absent ID	find P999	null	a missing pass gives null, no exception
T3 damaged lines present	after adding bad lines and P7, find P7	P7 found	damaged lines do not hide later valid records
T4 damaged record	find P4 (3 fields)	null	a damaged record is ignored
T5 list route 12	listByRoute(12)	P1 P2 P6 P7	list works; **weakness found:** P6 has a non-numeric amount but is still listed (only the field count is checked)

B1.2 Compare with the Part A design (1)

Differences from the A1 pseudocode: the implementation uses the 5-field record of the assignment (the A1 design had 7 fields); it uses Files.newBufferedReader/Writer (NIO) instead of FileReader/BufferedReader, which uses UTF-8 and closes through try-with-resources; the damaged-line check in the code only counts fields, while the design also validates numeric fields – test T5 showed that this is not enough, so the stricter validation was added in B2/B4. I also split with *split(",", -1)* (limit −1) so that trailing empty fields are not dropped and a line such as “a,b,,,” is counted correctly.

B1.3 Measure the effect of buffering (2)

File size	read() calls (unbuffered)	Unbuffered FileInputStream	BufferedInputStream (8 KB)	byte[64K]	BufferedReader	readAllLines
1 MB	1,048,600	237 ms	21 ms	1.5 ms	9 ms	7 ms
10 MB	10,485,790	2,520 ms	185 ms	4.4 ms	40 ms	28 ms
50 MB	52,428,810	12,020 ms	779 ms	17.5 ms	76 ms	325 ms

Compared with the prediction (A2.1): predicted 52.4 million calls and about 10 s for 50 MB – observed 52,428,810 calls and 12.0 s, time linear in the size (237 ms → 2.5 s → 12 s for 1 → 10 → 50 MB): confirmed. Buffering cut the time 11–15× (BufferedInputStream still makes one read() call per byte, but from memory) and the block read cut it by 700× (12,020 → 17.5 ms) because both the system calls and the per-byte method calls disappear. (The unbuffered 50 MB result is a single run because it takes 12 s.) readAllLines was fast (325 ms) but needs the whole file in memory.

B1.4 Measure growth (2)

Records	Find (file scan, last record)	HashMap lookup	HashMap build (one scan)
1,000	0.68 ms	0.001 ms	3.8 ms
10,000	2.2 ms	0.0002 ms	11.1 ms
50,000	10.5 ms	0.0002 ms	34.0 ms
100,000	17.4 ms	0.0001 ms	28.0 ms

Trend: the time to find a pass grows linearly with the file size (1,000 → 100,000 records, 100× more data → about 26× more time; the first value is inflated by JIT warm-up) – every search reads on average half of the file, in the worst case all of it: O(n). An in-memory HashMap answers in a fraction of a microsecond independent of n.

Alternative concept: read the file once into a HashMap<String, String[]>

`Map<String,String[]> m = new HashMap<>(); for each valid line: m.put(f[0], f).` **q searches on n passes:** scan every time = **O(q × n)**; HashMap = **O(n + q)** (one scan to build, O(1) per search). With the measurements: q = 1,000 searches on 100,000 passes ≈ 1,000 × 17.4 ms = 17.4 s with scans, versus 28 ms (build) + 1,000 × 0.0001 ms ≈ 28 ms with the map – the map wins after only 2 searches. **Memory:** scan approach = O(1) (one line and the buffer); HashMap = O(n): each record is a String[5] plus 5 Strings plus a map entry, roughly 300–400 bytes per record, i.e. about 30–40 MB for 100,000 passes (estimate, not measured). The map must also be reloaded or updated when a pass is added, and it is lost when the program ends.

TASK B2 – Engineer an Adversarial Input (5 marks)

Source file: B2Adversarial.java

```
import java.io.*;
import java.nio.file.*;
import java.sql.*;
import java.util.*;
import java.util.regex.Pattern;
public class B2Adversarial {
    static boolean unsafeLogin(Connection c, String id, String pwd) throws SQLException {
        String sql = "SELECT * FROM users WHERE id='" + id + "' AND pwd='" + pwd + "'";
        System.out.println("      SQL sent: " + sql);
        try (Statement st = c.createStatement(); ResultSet rs = st.executeQuery(sql)) { return rs.next(); }
    }
    static boolean safeLogin(Connection c, String id, String pwd) throws SQLException {
        try (PreparedStatement ps = c.prepareStatement("SELECT * FROM users WHERE id=? AND pwd=?")) {
            ps.setString(1, id); ps.setString(2, pwd);
            try (ResultSet rs = ps.executeQuery()) { return rs.next(); }
        }
    }
    static final Pattern SAFE = Pattern.compile("[A-Za-z0-9_]{1,20}");
    static boolean regexLogin(Connection c, String id, String pwd) throws SQLException {       // weaker alternative:
        validate then concatenate
        if (!SAFE.matcher(id).matches() || !SAFE.matcher(pwd).matches()) return false;          // also rejects legitimate
            passwords like "p@ss word!"
        try (Statement st = c.createStatement(); ResultSet rs = st.executeQuery("SELECT * FROM users WHERE id='" + id + "'
            AND pwd='" + pwd + "'")) { return rs.next(); }
    }
    static String escapeQuotes(String s) { return s.replace("'", "''"); }

    public static void main(String[] args) throws Exception {
        Path db = Files.createTempFile("b2", ".db");
        try (Connection c = DriverManager.getConnection("jdbc:sqlite:" + db); Statement st = c.createStatement()) {
            st.execute("CREATE TABLE users(id TEXT PRIMARY KEY, pwd TEXT)");
            st.execute("INSERT INTO users VALUES ('admin','S3cret'), ('arun','pass123')");
            String[][] tests = {
                {"valid login", "arun", "pass123"},
                {"wrong password", "arun", "wrong"},
                {"INJECTION: ' OR '1'='1", "arun", "x' OR '1'='1"},
                {"INJECTION: comment out password", "admin'--", "anything"},
                {"INJECTION: no valid user at all", "nobody' OR '1'='1' --", "zzz"},
            };
            System.out.println("== B2.1 unsafe (string concatenation) vs PreparedStatement ==");
            for (String[] t : tests) {
                System.out.println("  " + t[0] + "   id=[" + t[1] + "]  pwd=[" + t[2] + "]");
                boolean u = unsafeLogin(c, t[1], t[2]);
                boolean s = safeLogin(c, t[1], t[2]);
                System.out.println("      unsafe accepted=" + u + "   PreparedStatement accepted=" + s);
            }
            System.out.println("\n== Weaker defences ==");
            System.out.println("  regex validation: injection pwd rejected = " + !regexLogin(c, "arun", "x' OR '1'='1") + ";
                legitimate password 'p@ss word!' rejected too = " + !regexLogin(c, "arun", "p@ss word!"));
            String evil = "x' OR '1'='1";
            System.out.println("  quote escaping  : escaped pwd=[" + escapeQuotes(evil) + "] -> accepted=" + unsafeLogin(c,
                "arun", escapeQuotes(evil)));
            System.out.println("  (escaping works only for this one pattern; it depends on the DB's quoting rules, e.g. MySQL
                 also treats backslash as an escape character)");
            long t = System.nanoTime(); for (int i = 0; i < 20000; i++) SAFE.matcher("arun" + i).matches(); double rx =
                (System.nanoTime() - t) / 2e4;
            t = System.nanoTime(); try (PreparedStatement ps = c.prepareStatement("SELECT * FROM users WHERE id=? AND
                pwd=?")) { for (int i = 0; i < 20000; i++) { ps.setString(1, "arun"); ps.setString(2, "pass123"); try
                (ResultSet rs = ps.executeQuery()) { rs.next(); } } } double ps = (System.nanoTime() - t) / 2e4;
            System.out.printf("  cost per call: regex validation %.0f ns; reusing one PreparedStatement for a whole query
                %.0f ns (validation is cheap but gives no guarantee)%n", rx, ps);
        }
        System.out.println("\n== B2.2 damaged bus-pass file ==");
        Path f = Files.createTempFile("damaged", ".csv");
        Files.writeString(f, "P1,192311001,Arun,12,4500.0\n" + "P2,192311002,Meena,12\n" + "P3,192311003,Ravi,7,abc\n" + "\n"
             + "P5,192311005,Lata,12,3900.0,EXTRA\n" + "P6,192311006,Kumar,x,3000\n" + "P7,192311007,Divya,9,3800.0\n");
        PassStore st = new PassStore(f);
        System.out.println("  Original PassStore.find (checks only the number of fields):");
        for (String id : new String[]{"P1", "P2", "P3", "P5", "P6", "P7"}) System.out.println("    find(" + id + ") -> " +
            Arrays.toString(st.find(id)));
        System.out.println("  Strict parser (also validates route and amount) - report of every line:");
        int lineNo = 0, ok = 0;
        for (String line : Files.readAllLines(f)) {
            lineNo++; String why = null; String[] x = line.split(",", -1);
            if (line.isBlank()) why = "empty line";
            else if (x.length != 5) why = "wrong field count (" + x.length + ")";
            else try { Integer.parseInt(x[3]); Double.parseDouble(x[4]); } catch (NumberFormatException e) { why = "non-
                numeric route/amount"; }
            if (why == null) ok++; else System.out.println("    line " + lineNo + " REJECTED: " + why + "  -> " + line);
        }
        System.out.println("    valid lines: " + ok + " of " + lineNo);
    }
}
```

Execution and output (local test database created for this assignment):

```
$ javac -cp sqlite-jdbc.jar B2Adversarial.java PassStore.java && java -cp .:sqlite-jdbc.jar B2Adversarial
== B2.1 unsafe (string concatenation) vs PreparedStatement ==
  valid login   id=[arun]  pwd=[pass123]
      SQL sent: SELECT * FROM users WHERE id='arun' AND pwd='pass123'
      unsafe accepted=true   PreparedStatement accepted=true
  wrong password   id=[arun]  pwd=[wrong]
```

```
      SQL sent: SELECT * FROM users WHERE id='arun' AND pwd='wrong'
      unsafe accepted=false   PreparedStatement accepted=false
  INJECTION: ' OR '1'='1   id=[arun]  pwd=[x' OR '1'='1]
      SQL sent: SELECT * FROM users WHERE id='arun' AND pwd='x' OR '1'='1'
      unsafe accepted=true   PreparedStatement accepted=false
  INJECTION: comment out password   id=[admin'--]  pwd=[anything]
      SQL sent: SELECT * FROM users WHERE id='admin'--' AND pwd='anything'
      unsafe accepted=true   PreparedStatement accepted=false
  INJECTION: no valid user at all   id=[nobody' OR '1'='1' --]  pwd=[zzz]
      SQL sent: SELECT * FROM users WHERE id='nobody' OR '1'='1' --' AND pwd='zzz'
      unsafe accepted=true   PreparedStatement accepted=false
== Weaker defences ==
  regex validation: injection pwd rejected = true; legitimate password 'p@ss word!' rejected too = true
      SQL sent: SELECT * FROM users WHERE id='arun' AND pwd='x'' OR ''1''='1'
  quote escaping  : escaped pwd=[x'' OR ''1''='1] -> accepted=false
  (escaping works only for this one pattern; it depends on the DB's quoting rules, e.g. MySQL also treats backslash as an
      escape character)
  cost per call: regex validation 1754 ns; reusing one PreparedStatement for a whole query 11475 ns (validation is cheap but
      gives no guarantee)

== B2.2 damaged bus-pass file ==
  Original PassStore.find (checks only the number of fields):
    find(P1) -> [P1, 192311001, Arun, 12, 4500.0]
    find(P2) -> null
    find(P3) -> [P3, 192311003, Ravi, 7, abc]
    find(P5) -> null
    find(P6) -> [P6, 192311006, Kumar, x, 3000]
    find(P7) -> [P7, 192311007, Divya, 9, 3800.0]
  Strict parser (also validates route and amount) - report of every line:
    line 2 REJECTED: wrong field count (4)  -> P2,192311002,Meena,12
    line 3 REJECTED: non-numeric route/amount  -> P3,192311003,Ravi,7,abc
    line 4 REJECTED: empty line  ->
    line 5 REJECTED: wrong field count (6)  -> P5,192311005,Lata,12,3900.0,EXTRA
    line 6 REJECTED: non-numeric route/amount  -> P6,192311006,Kumar,x,3000
    valid lines: 2 of 7
```

Adversarial input	Unsafe (string concatenation)	PreparedStatement
pwd = `x' OR '1'='1`	**accepted**	rejected
id = `admin'--`, pwd = anything	**accepted** (password comment-out)	rejected
id = `nobody' OR '1'='1' --` (no such user)	**accepted**	rejected
valid login / wrong password	accepted / rejected	accepted / rejected

What each adversarial input reveals about the original design

- **Injection inputs:** the unsafe design lets the user change the *structure* of the SQL, i.e. input is mixed with code – authentication can be bypassed completely, even for a user that does not exist; with other statements data could be read or destroyed. The PreparedStatement treats the same text as a value that matches no password.
- **Wrong number of fields (lines 2 and 5) and the empty line (line 4):** the store skips them – the design is tolerant of structural damage and the search continues.
- **Non-numeric amount (P3: *abc*) and non-numeric route (P6: *x*):** the original find() **returned these records as if they were valid** – the original design checks only the shape, not the content, so bad data would later crash arithmetic or be inserted in a numeric column. The strict parser rejects them (line report above: 5 rejected, 2 of 7 lines valid).
- Lesson: validate at the boundary (every line read, every string used in SQL); never trust that a line which "looks" right is right.

Alternative concept: escape quotes or validate with a regular expression instead of PreparedStatement

Escaping (replace ' by ''): it stopped this one attack (accepted = false) but depends on the database's quoting rules – MySQL also treats the backslash as an escape character, so input ending in a backslash or multibyte-character tricks can break it; one missed field is a hole. **Regex validation** (letters, digits, underscore only): rejected the attack, but also rejected a legitimate password such as *p@ss word!* (shown above) and still concatenates SQL – any gap in the pattern is exploitable. Both are **weaker** because they try to recognise bad input (a blacklist or a narrow whitelist) instead of separating code and data. **Time cost per query:** validation O(L) in the input length: measured 1.75 µs per call (regex); a PreparedStatement binds parameters in O(L) too; the measured 11.5 µs per execution includes running the query; preparing once and re-executing is cheaper than parsing SQL each time on a real server. The cost difference is irrelevant compared with the security difference.

TASK B3 – Statement vs PreparedStatement vs Batch (8 marks)

SQL script for the test database (MySQL form; the SQLite form used for the runs only differs in the types):

SQL script (create_tables.sql)

```
-- MySQL
CREATE DATABASE IF NOT EXISTS transport_test;
USE transport_test;
CREATE TABLE bus_pass (
    pass_id  VARCHAR(10) PRIMARY KEY,
    reg_no   VARCHAR(9)  NOT NULL,
    name     VARCHAR(60) NOT NULL,
    route_no INT         NOT NULL,
    amount   DECIMAL(8,2) NOT NULL
);
CREATE TABLE users (id VARCHAR(20) PRIMARY KEY, pwd VARCHAR(60));   -- for task B2
-- URL for the program: jdbc:mysql://localhost:3306/transport_test?rewriteBatchedStatements=true
```

Complete the Code – PassLoader (TODO 1 to TODO 3)

Source file: PassLoader.java

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.List;

public class PassLoader {
    /** Loads all passes in one transaction, in batches of 1,000 rows.
     * Returns the number of rows loaded. */
    public static int loadAll(Connection con, List<String[]> passes)
            throws SQLException {
        String sql = "INSERT INTO bus_pass(pass_id, reg_no, name, route_no, amount) "
                + "VALUES (?, ?, ?, ?, ?)";
        boolean oldAutoCommit = con.getAutoCommit();
        con.setAutoCommit(false);
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            int n = 0;
            for (String[] p : passes) {
                ps.setString(1, p[0]); ps.setString(2, p[1]); ps.setString(3, p[2]);   // TODO 1 completed
                ps.setInt(4, Integer.parseInt(p[3]));
                ps.setDouble(5, Double.parseDouble(p[4]));
                ps.addBatch();
                n++;
                if (n % 1000 == 0) ps.executeBatch();
            }
            ps.executeBatch();
            con.commit();                                                       // TODO 2 completed
            return n;
        } catch (SQLException | RuntimeException e) {
            con.rollback();                                                     // TODO 3 completed
            throw e;
        } finally {
            con.setAutoCommit(oldAutoCommit);
        }
    }
}
```

Source file: B3Bench.java

```
import java.nio.file.*;
import java.sql.*;
import java.util.*;

public class B3Bench {
    static final String DDL = "CREATE TABLE bus_pass(pass_id TEXT PRIMARY KEY, reg_no TEXT NOT NULL, name TEXT NOT NULL,
        route_no INTEGER NOT NULL, amount REAL NOT NULL)";
    static Connection fresh() throws Exception {
        Path p = Files.createTempFile("b3", ".db"); Connection c = DriverManager.getConnection("jdbc:sqlite:" + p);
        try (Statement s = c.createStatement()) { s.execute(DDL); } return c;
    }
    static List<String[]> gen(int n) { List<String[]> l = new ArrayList<>(n); for (int i = 0; i < n; i++) l.add(new
        String[]{"P" + i, "19231" + (10000 + i), "Student" + i, String.valueOf(i % 20), String.valueOf(3000 + i % 900)});
        return l; }
    static int count(Connection c) throws SQLException { try (Statement s = c.createStatement(); ResultSet r =
        s.executeQuery("SELECT COUNT(*) FROM bus_pass")) { r.next(); return r.getInt(1); } }

    static double viaStatement(Connection c, List<String[]> d) throws SQLException {
        long t = System.nanoTime(); try (Statement s = c.createStatement()) { for (String[] p : d) s.executeUpdate("INSERT
            INTO bus_pass VALUES ('" + p[0] + "','" + p[1] + "','" + p[2] + "'," + p[3] + "," + p[4] + ")"); } return
            (System.nanoTime() - t) / 1e6; }
    static double viaPrepared(Connection c, List<String[]> d) throws SQLException {
        long t = System.nanoTime(); try (PreparedStatement ps = c.prepareStatement("INSERT INTO bus_pass VALUES
            (?,?,?,?,?)")) { for (String[] p : d) { ps.setString(1, p[0]); ps.setString(2, p[1]); ps.setString(3, p[2]);
            ps.setInt(4, Integer.parseInt(p[3])); ps.setDouble(5, Double.parseDouble(p[4])); ps.executeUpdate(); } } return
            (System.nanoTime() - t) / 1e6; }
    static double viaBatchAuto(Connection c, List<String[]> d) throws SQLException {
        long t = System.nanoTime(); try (PreparedStatement ps = c.prepareStatement("INSERT INTO bus_pass VALUES
            (?,?,?,?,?)")) { int n = 0; for (String[] p : d) { ps.setString(1, p[0]); ps.setString(2, p[1]); ps.setString(3,
            p[2]); ps.setInt(4, Integer.parseInt(p[3])); ps.setDouble(5, Double.parseDouble(p[4])); ps.addBatch(); if (++n %
            1000 == 0) ps.executeBatch(); } ps.executeBatch(); } return (System.nanoTime() - t) / 1e6; }
    static double viaMultiRow(Connection c, List<String[]> d) throws SQLException {
        long t = System.nanoTime(); c.setAutoCommit(false);
        for (int i = 0; i < d.size(); i += 100) {                                  // SQLite limits host parameters, so 100
            rows (500 params) per statement
            int k = Math.min(100, d.size() - i); StringBuilder sb = new StringBuilder("INSERT INTO bus_pass VALUES ");
            for (int j = 0; j < k; j++) sb.append(j == 0 ? "" : ",").append("(?,?,?,?,?)");
            try (PreparedStatement ps = c.prepareStatement(sb.toString())) { int q = 1; for (int j = 0; j < k; j++) {
                String[] p = d.get(i + j); ps.setString(q++, p[0]); ps.setString(q++, p[1]); ps.setString(q++, p[2]);
                ps.setInt(q++, Integer.parseInt(p[3])); ps.setDouble(q++, Double.parseDouble(p[4])); } ps.executeUpdate(); }
        }
        c.commit(); c.setAutoCommit(true); return (System.nanoTime() - t) / 1e6; }
    public static void main(String[] args) throws Exception {
        System.out.println("== B3.1 correctness on 10 records (each approach into its own table copy) ==");
        List<String[]> ten = gen(10);
        for (String name : new String[]{"Statement", "PreparedStatement", "Batch"}) {
            try (Connection c = fresh()) {
                if (name.equals("Statement")) viaStatement(c, ten); else if (name.equals("PreparedStatement")) viaPrepared(c,
                     ten); else PassLoader.loadAll(c, ten);
                System.out.println("  " + name + ": rows=" + count(c));
                if (name.equals("Batch")) try (Statement s = c.createStatement(); ResultSet r = s.executeQuery("SELECT * FROM
                     bus_pass ORDER BY route_no, pass_id LIMIT 3")) {
                    ResultSetMetaData md = r.getMetaData();
                    for (int i = 1; i <= md.getColumnCount(); i++) System.out.printf("     column %d: %-9s declared type %-8s
                         JDBC type %-8s nullable=%s%n", i, md.getColumnName(i), md.getColumnTypeName(i),
                        JDBCType.valueOf(md.getColumnType(i)), md.isNullable(i) == 0 ? "NO" : "yes");
                    while (r.next()) System.out.println("     " + r.getString(1) + " | " + r.getString(2) + " | " +
                        r.getString(3) + " | " + r.getInt(4) + " | " + r.getDouble(5));
                }
            }
        }
        System.out.println("\n== B3.2 time to insert n rows (ms, SQLite file database, autocommit ON for Statement and
            PreparedStatement; batch inside the loader's transaction) ==");
        System.out.printf("%-9s %-12s %-18s %-22s %-18s%n", "n", "Statement", "PreparedStatement", "PS + addBatch(1000)",
            "multi-row INSERT");
        for (int n : new int[]{1000, 10000, 100000}) {
            List<String[]> d = gen(n); double s = -1, p = -1, b, m;
            if (n <= 10000) { try (Connection c = fresh()) { s = viaStatement(c, d); } try (Connection c = fresh()) { p =
                viaPrepared(c, d); } }
            else { try (Connection c = fresh()) { s = viaStatement(c, d.subList(0, 10000)) * 10; } try (Connection c =
                fresh()) { p = viaPrepared(c, d.subList(0, 10000)) * 10; } }
            try (Connection c = fresh()) { long t = System.nanoTime(); PassLoader.loadAll(c, d); b = (System.nanoTime() - t)
                / 1e6; if (count(c) != n) throw new IllegalStateException(); }
            try (Connection c = fresh()) { m = viaMultiRow(c, d); }
            System.out.printf("%-9d %-12s %-18s %-22.0f %-18.0f%s%n", n, String.format("%.0f", s), String.format("%.0f", p),
                b, m, n > 10000 ? "   (* Statement/PS at 100,000 extrapolated from a measured 10,000-row run x10 because
                autocommit makes each row an fsync)" : "");
        }
        System.out.println("\n== B3.3 auto-commit vs single transaction, 10,000 rows, failure forced at row 5,000 (invalid
            amount 'ABC') ==");
        List<String[]> d = gen(10000); d.set(4999, new String[]{"P4999", "x", "x", "1", "ABC"});
        try (Connection c = fresh()) {                                        // auto-commit ON, batch of 1,000 committed as
            they go
            c.setAutoCommit(true); int done = 0;
            try (PreparedStatement ps = c.prepareStatement("INSERT INTO bus_pass VALUES (?,?,?,?,?)")) {
                for (String[] p : d) { ps.setString(1, p[0]); ps.setString(2, p[1]); ps.setString(3, p[2]); ps.setInt(4,
                    Integer.parseInt(p[3])); ps.setDouble(5, Double.parseDouble(p[4])); ps.executeUpdate(); done++; }
            } catch (NumberFormatException e) { System.out.println("  auto-commit: failure after " + done + " rows (" +
                e.getMessage() + ")"); }
```

```
            System.out.println("  auto-commit: rows left in table = " + count(c) + "   <-- partial load, table is
                inconsistent");
        }
        try (Connection c = fresh()) {
            try { PassLoader.loadAll(c, d); } catch (RuntimeException e) { System.out.println("  transaction: loadAll threw "
                 + e.getClass().getSimpleName() + ": " + e.getMessage()); }
            System.out.println("  transaction: rows left in table = " + count(c) + "   <-- all-or-nothing");
        }
        System.out.println("\n== autocommit ON vs one transaction, batch insert of 10,000 valid rows (ms) ==");
        List<String[]> ok = gen(10000);
        try (Connection c = fresh()) { c.setAutoCommit(true); System.out.printf("  batch, auto-commit ON  (executeBatch each
            1000) : %.0f ms%n", viaBatchAuto(c, ok)); }
        try (Connection c = fresh()) { long t = System.nanoTime(); PassLoader.loadAll(c, ok); System.out.printf("  batch,
            single transaction (PassLoader)           : %.0f ms%n", (System.nanoTime() - t) / 1e6); }
    }
}
```

Execution and output:

```
$ javac -cp sqlite-jdbc.jar PassLoader.java B3Bench.java && java -cp .:sqlite-jdbc.jar B3Bench
== B3.1 correctness on 10 records (each approach into its own table copy) ==
  Statement: rows=10
  PreparedStatement: rows=10
  Batch: rows=10
     column 1: pass_id   declared type TEXT     JDBC type VARCHAR  nullable=NO
     column 2: reg_no    declared type TEXT     JDBC type VARCHAR  nullable=yes
     column 3: name      declared type TEXT     JDBC type VARCHAR  nullable=yes
     column 4: route_no  declared type INTEGER  JDBC type INTEGER  nullable=yes
     column 5: amount    declared type REAL     JDBC type REAL     nullable=yes
     P0 | 1923110000 | Student0 | 0 | 3000.0
     P1 | 1923110001 | Student1 | 1 | 3001.0
     P2 | 1923110002 | Student2 | 2 | 3002.0

== B3.2 time to insert n rows (ms, SQLite file database, autocommit ON for Statement and PreparedStatement; batch inside the
    loader's transaction) ==
n         Statement    PreparedStatement  PS + addBatch(1000)    multi-row INSERT
1000      964          762                7                      11
10000     8047         7408               35                     23
100000    70469        71147              234                    316                  (* Statement/PS at 100,000 extrapolated
     from a measured 10,000-row run x10 because autocommit makes each row an fsync)

== B3.3 auto-commit vs single transaction, 10,000 rows, failure forced at row 5,000 (invalid amount 'ABC') ==
  auto-commit: failure after 4999 rows (For input string: "ABC")
  auto-commit: rows left in table = 4999   <-- partial load, table is inconsistent
  transaction: loadAll threw NumberFormatException: For input string: "ABC"
  transaction: rows left in table = 0   <-- all-or-nothing

== autocommit ON vs one transaction, batch insert of 10,000 valid rows (ms) ==
  batch, auto-commit ON  (executeBatch each 1000) : 6656 ms
  batch, single transaction (PassLoader)           : 34 ms
```

B3.1 Correctness on small data (2)

10 records inserted with each approach – each table holds 10 rows. SELECT with ResultSetMetaData reports the five columns: pass_id TEXT (VARCHAR, NOT NULL because it is the primary key), reg_no and name TEXT, route_no INTEGER, amount REAL; the rows read back equal the data inserted (P0 | 1923110000 | Student0 | 0 | 3000.0 ...). *Observation:* the SQLite driver reports reg_no and name as nullable although the DDL says NOT NULL (a driver limitation); in MySQL the metadata would also show VARCHAR(9) etc. with exact sizes and DECIMAL for amount.

B3.2 Increase the data size (2)

Rows	Statement	PreparedStatement	PS + addBatch/executeBatch (1000)	Multi-row INSERT (100 rows/statement)
1,000	964 ms	762 ms	7 ms	11 ms
10,000	8,047 ms	7,408 ms	35 ms	23 ms
100,000	70,469 ms*	71,147 ms*	234 ms	316 ms

*100,000-row values for Statement and PreparedStatement are **extrapolated (10,000-row run × 10)** because with auto-commit every row is a separate committed transaction (a disk flush), which makes the run take over a minute; the linear trend from 1,000 → 10,000 (8.4× and 9.7× for 10× the data) justifies it. The batch inside one transaction scales: 7 → 35 → 234 ms (about 1/300 of the time of the row-by-row approaches at 100,000 rows).

B3.3 Auto-commit vs transaction (2)

- **Same batch of 10,000 rows:** auto-commit ON 6,656 ms; single transaction (PassLoader) 34 ms – about 200× faster, because there is one commit instead of one per row (or per batch statement).
- **Failure forced at row 5,000 (amount “ABC”):** with auto-commit the first 4,999 rows were already committed – **table contains 4,999 rows** (a partial, inconsistent load, and you cannot tell which rows are missing without checking); with the transaction (loadAll) the NumberFormatException triggers rollback(): **0 rows** remain, the table is unchanged.

This is the required evidence for “insert a pass with an invalid amount half-way through the list and show that no rows remain in the table after the failure”: rows left in table = 0.

B3.4 Engineering trade-off (2)

(a) Nightly import of 100,000 passes: PreparedStatement with batches (1,000 rows per executeBatch) inside **one transaction** (PassLoader): fastest of the row-based methods, parameterised (safe), all-or-nothing, and with MySQL the connection option rewriteBatchedStatements=true turns the batch into multi-row inserts (LOAD DATA INFILE is faster still when the file is trusted and local). **(b) A single pass issued at the counter:** a plain PreparedStatement with the default auto-commit – one statement is already atomic, batching has nothing to gain, and the pass is committed (durable) immediately. Statement (concatenated SQL) is not recommended for either case: no speed advantage over PreparedStatement and exposed to SQL injection.

Alternative concept: one multi-row INSERT or MySQL LOAD DATA

Multi-row INSERT: `INSERT ... VALUES (?,?,?,?,?),(?,?,?,?,?),...` – one statement carries b rows, parsed once; measured 11 / 23 / 316 ms for 1,000 / 10,000 / 100,000 rows, the same order as the batch (7 / 35 / 234 ms); the limit is the maximum statement size / number of parameters (SQLite 32,766 parameters; MySQL max_allowed_packet). **LOAD DATA INFILE** (not executed here – no MySQL server): the server reads the file directly, no per-row parsing or network round trip; for 100,000 rows it typically takes well below a second, but needs file access/privileges on the server and bypasses PreparedStatement validation, so the data must be validated first (B4). **Total time for n rows:** row-by-row O(n) round trips; batch O(n/b) round trips + O(n) work; multi-row O(n/b); LOAD DATA one operation. **Memory for a batch of b rows:** batch = b parameter sets (O(b) objects in the driver); multi-row = one SQL string of length O(b) plus b×5 parameters; LOAD DATA = O(1) on the client (streaming) – so b must be bounded (1,000 rows here) for all of them.

TASK B4 – Final Engineering Decision (5 marks)

Source file: B4Migrate.java

```
import java.io.*;
import java.nio.file.*;
import java.sql.*;
import java.util.*;

public class B4Migrate {
    record Failure(int line, String reason, String text) {}

    /** Reads the pass file, validates every line, loads valid rows in batches inside ONE transaction. */
    static int migrate(Path file, Connection con, List<Failure> failures) throws Exception {
        List<String[]> valid = new ArrayList<>(); Set<String> seen = new HashSet<>(); int ln = 0;
        try (BufferedReader r = Files.newBufferedReader(file)) {
            for (String line; (line = r.readLine()) != null; ) {
                ln++;
                if (line.isBlank()) { failures.add(new Failure(ln, "empty line", line)); continue; }
                String[] f = line.split(",", -1);
                if (f.length != 5) { failures.add(new Failure(ln, "wrong field count (" + f.length + ")", line)); continue; }
                try { Integer.parseInt(f[3].trim()); Double.parseDouble(f[4].trim()); }
                catch (NumberFormatException e) { failures.add(new Failure(ln, "non-numeric route or amount", line));
                    continue; }
                if (!f[1].matches("\\d{9}")) { failures.add(new Failure(ln, "register number not 9 digits", line)); continue;
                     }
                if (!seen.add(f[0])) { failures.add(new Failure(ln, "duplicate pass ID " + f[0], line)); continue; }
                valid.add(f);
            }
        }
        return PassLoader.loadAll(con, valid);
    }

    public static void main(String[] args) throws Exception {
        Path in = Files.createTempFile("passes", ".csv");
        try (BufferedWriter w = Files.newBufferedWriter(in)) {
            for (int i = 1; i <= 5000; i++) { w.write("P" + i + ",19231" + String.format("%04d", i % 10000) + ",Student" + i
                + "," + (i % 12) + "," + (3000 + i % 800) + ".0"); w.newLine(); }
            w.write("P6000,1923,Short,5,100\nP6001,192310001,Bad,x,100\n\nP6002,192310002,Few,5\nP1,192319999,Dup,3,500\n");
                    // five deliberately damaged lines
        }
        Path db = Files.createTempFile("transport", ".db");
        try (Connection con = DriverManager.getConnection("jdbc:sqlite:" + db)) {
            try (Statement s = con.createStatement()) { s.execute(B3Bench.DDL); }
            List<Failure> bad = new ArrayList<>();
            long t = System.nanoTime(); int loaded = migrate(in, con, bad); double ms = (System.nanoTime() - t) / 1e6;
            System.out.printf("== B4.1 migration ==%nlines read: %d   rows loaded: %d   failed validation: %d   time %.0f
                ms%n", loaded + bad.size(), loaded, bad.size(), ms);
            for (Failure f : bad) System.out.println("  line " + f.line() + ": " + f.reason() + "   -> [" + f.text() + "]");
            try (Statement s = con.createStatement(); ResultSet r = s.executeQuery("SELECT COUNT(*) FROM bus_pass")) {
                r.next(); System.out.println("rows in bus_pass: " + r.getInt(1)); }
            System.out.println("\n== B4.2 DatabaseMetaData ==");
            DatabaseMetaData md = con.getMetaData();
            System.out.println("product        : " + md.getDatabaseProductName() + " " + md.getDatabaseProductVersion());
            System.out.println("driver         : " + md.getDriverName() + " " + md.getDriverVersion());
            System.out.println("JDBC version   : " + md.getJDBCMajorVersion() + "." + md.getJDBCMinorVersion());
            System.out.println("URL            : " + md.getURL().replaceAll("/tmp/\\S+", "<temp file>"));
            System.out.println("supports batch updates: " + md.supportsBatchUpdates() + " | supports transactions: " +
                md.supportsTransactions());
            try (ResultSet tb = md.getTables(null, null, "%", new String[]{"TABLE"})) { while (tb.next())
                System.out.println("table          : " + tb.getString("TABLE_NAME")); }
            try (ResultSet cols = md.getColumns(null, null, "bus_pass", "%")) { while (cols.next()) System.out.println("
                column " + cols.getString("COLUMN_NAME") + " " + cols.getString("TYPE_NAME")); }
            try (ResultSet pk = md.getPrimaryKeys(null, null, "bus_pass")) { while (pk.next()) System.out.println("primary
                key    : " + pk.getString("COLUMN_NAME")); }
        }
        System.out.println("\n== B4.3 scale estimate from measurements ==");
        System.out.println("100x passes = 800,000 passes. File scan measured 17.4 ms/search at 100,000 records -> about 140
            ms/search at 800,000 (linear), 12 counters x many searches/min is unsafe; HashMap needs the whole file in RAM and
             cannot be shared by 12 campuses.");
    }
}
```

Execution and output:

```
$ javac -cp sqlite-jdbc.jar B4Migrate.java PassLoader.java B3Bench.java && java -cp .:sqlite-jdbc.jar B4Migrate
== B4.1 migration ==
lines read: 5005   rows loaded: 5000   failed validation: 5   time 141 ms
  line 5001: register number not 9 digits   -> [P6000,1923,Short,5,100]
  line 5002: non-numeric route or amount   -> [P6001,192310001,Bad,x,100]
  line 5003: empty line   -> []
  line 5004: wrong field count (4)   -> [P6002,192310002,Few,5]
  line 5005: duplicate pass ID P1   -> [P1,192319999,Dup,3,500]
rows in bus_pass: 5000

== B4.2 DatabaseMetaData ==
product        : SQLite 3.36.0
driver         : SQLite JDBC 3.36.0.3
JDBC version   : 2.1
URL            : jdbc:sqlite:<temp file>
supports batch updates: true | supports transactions: true
table          : bus_pass
  column pass_id TEXT
```

```
  column reg_no TEXT
  column name TEXT
  column route_no INTEGER
  column amount REAL
primary key    : pass_id
== B4.3 scale estimate from measurements ==
100x passes = 800,000 passes. File scan measured 17.4 ms/search at 100,000 records -> about 140 ms/search at 800,000
    (linear), 12 counters x many searches/min is unsafe; HashMap needs the whole file in RAM and cannot be shared by 12
    campuses.
```

B4.1 Migration with failure report (2)

The program reads the pass file line by line (BufferedReader), validates each line (field count, numeric route and amount, 9-digit register number, duplicate pass ID), collects the valid rows and loads them with PassLoader.loadAll (batches of 1,000 in one transaction). For a file of 5,005 lines with 5 deliberately damaged lines it loaded **5,000 rows**, rejected **5 lines and reported each with its line number and reason**, in 141 ms; the table contains 5,000 rows. If the database load itself fails, the transaction rolls back and nothing is migrated.

B4.2 Inspect the database (1)

DatabaseMetaData reported: product SQLite 3.36.0; driver SQLite JDBC 3.36.0.3; JDBC 2.1; table bus_pass with its columns; primary key pass_id; supportsBatchUpdates = true; supportsTransactions = true. **Why this is useful when the application moves to another server:** the program can check that the product/version and driver are the expected ones, that the required tables and columns exist, and that features it depends on (transactions, batch updates, supported types) are available, and can choose the right SQL dialect or give a clear error before running the migration – instead of failing half-way with a cryptic SQL error.

B4.3 Production decision (2)

Requirements at 100× scale: 12 campuses, about 800,000 passes, several counters at the same time, fast search by pass ID, and no lost or corrupted passes.

Requirement	Text file	Serialized file	Database (JDBC)
Search by pass ID (measured)	linear: 17.4 ms at 100,000 → about 140 ms at 800,000 per search, per counter	needs the whole object graph in memory; load of 100,000 objects 90 ms, but every counter would need its own copy	index on pass_id: O(log n), independent of file scanning
Several counters at once	concurrent appends can interleave or overwrite; no locking	one writer must rewrite the whole file; concurrent writers lose updates	transactions and locking built in; many connections
Safety (no lost update on failure)	partial last line possible after a power failure	a crash while writing can corrupt the whole file	atomic transactions (rollback shown in B3.3: 0 rows after failure), constraints (primary key)
Bulk loading	easy	easy	batch in one transaction: 100,000 rows in 234 ms (measured)
Sharing between 12 campuses	files must be copied/synchronised	same	one central server

Decision: use a relational database (MySQL) accessed through JDBC with PreparedStatement, a Dao layer and transactions. Evidence: file scans grow linearly (17.4 ms per search at 100,000 records, extrapolated 140 ms at 800,000) while an indexed query does not; the transaction experiment (rows left 4,999 with auto-commit vs 0 with a transaction) shows the safety needed for passes and money; the batch experiment shows that the one-time migration costs well under a second per 100,000 rows. A text file remains useful for the export/import format of the nightly job, and serialization is not suitable for shared multi-user data.

Alternative concept: each campus keeps its own serialized file

Solution: twelve files (one per campus), each loaded into a collection at start-up. **Time to search all passes** (a pass issued at another campus, a university-wide report): every file must be opened, deserialized and searched – 12 × O(n/12) = O(n) in total, plus 12 separate loads (measured load: 90 ms for 100,000 objects, so about 0.7 s for 800,000) – and if the files are on different machines also 12 network transfers; one central database with an index on pass_id does the same in O(log n) on the server. **Space:** total data O(n) in both, but each campus program holds its whole file in memory (about 1/12 of n objects) and serialized files are about 1.5× larger than the equivalent text (measured, A4); copies in several places mean duplicates of data. **Concurrent counters:** with serialized files each campus process owns its file, so counters inside one campus must share one in-memory object guarded by synchronized/locks and the whole file is rewritten to save (no incremental update); updates from other campuses are invisible until files are exchanged. With one database the DBMS handles concurrency through transactions, row locks and unique keys, so a pass ID can never be issued twice by two counters.

SUMMARY OF PROGRAMS EXECUTED

Program	Task	Result
PassStore.java	B1 Complete the Code	Output equals expected output
B1Tests.java	B1.1, B1.3, B1.4	Tests, buffering table, growth table
B2Adversarial.java	B2	Injection accepted by unsafe, rejected by PreparedStatement; damaged file report
PassLoader.java, B3Bench.java	B3	Batch+transaction 234 ms/100,000 rows; rollback leaves 0 rows
B4Migrate.java	B4	5,000 loaded, 5 failures reported, metadata printed
SerialBench.java, ser/ demo	A4	serialVersionUID behaviour and size/time comparison

Note: timings depend on the machine (and, for JDBC, on SQLite instead of MySQL); re-run the programs on your own computer, add your own screenshots next to the outputs and replace the figures with your measurements. Keep one folder per task in the GitHub repository with at least one commit per task, as required.